

clustering

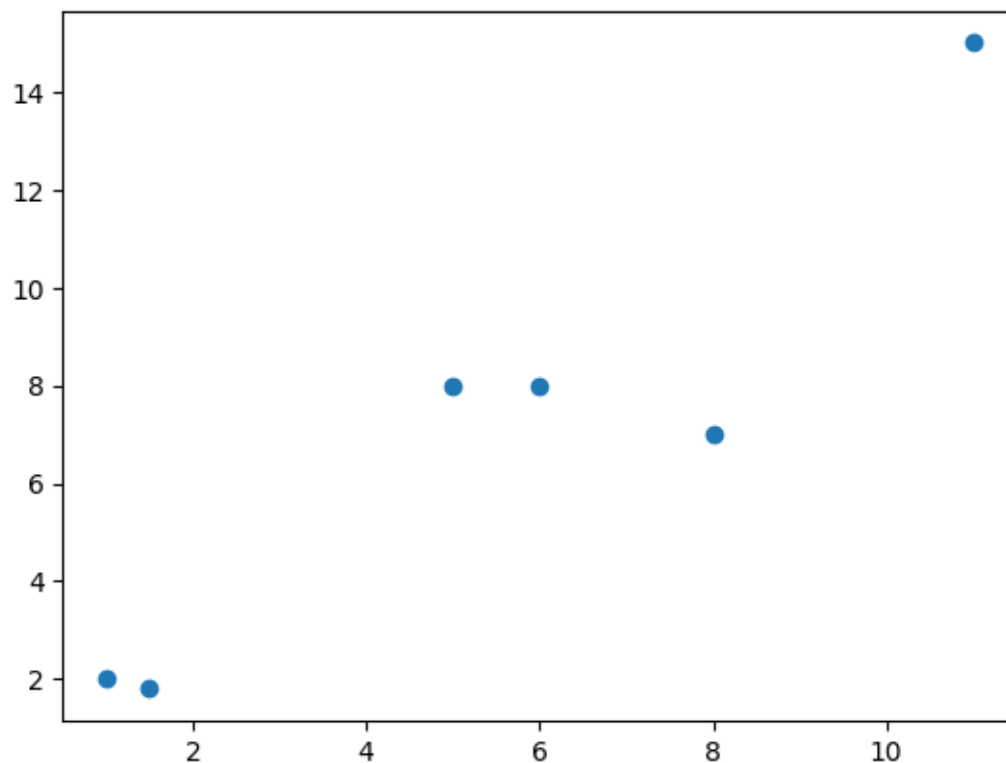
April 1, 2025

```
[ ]: #K-means Clustering Algorithm
```

Divides the data into K clusters based on the mean distance. Assigns each point to the nearest centroid and recalculates centroids iteratively.

Steps: 1. Choose K (number of clusters) 2. Initialize K centroids randomly 3. Assign each data point to the nearest centroid

```
[ ]: x=[1,5,1.5,6,8,11]  
     y=[2,8,1.8,8,7,15]  
     plt.scatter(x,y)  
     plt.show()
```



```
[ ]: #perform K-means clustering
from sklearn.cluster import KMeans
kmeans=KMeans(n_clusters=2)
kmeans.fit(X)
```

```
[ ]: KMeans(n_clusters=2)
```

```
[ ]: from sklearn.preprocessing import StandardScaler

np.random.seed(42)
data= {
    "Customer ID":range(1,201),
    "Annual Income": np.random.randint(15,135,200),
    "Spending Score": np.random.randint(1,101,200)
}
df=pd.DataFrame(data)
print(df)

X=df[['Annual Income','Spending Score']]
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)

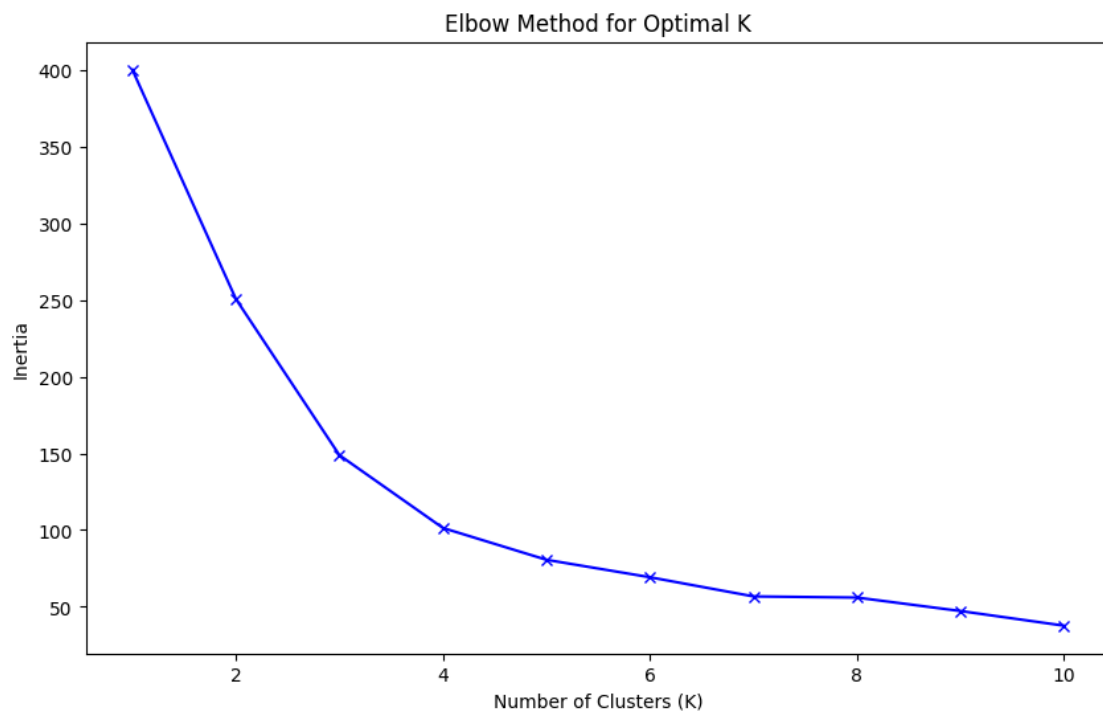
inertia=[]
K_range=range(1,11)
for k in K_range:
    kmeans=KMeans(n_clusters=k,random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
plt.figure(figsize=(10,6))
plt.plot(K_range,inertia,'bx-')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal K')
plt.show()
# Apply KMeans with the chosen K (let's assume 4 clusters based on the elbow
method)
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)

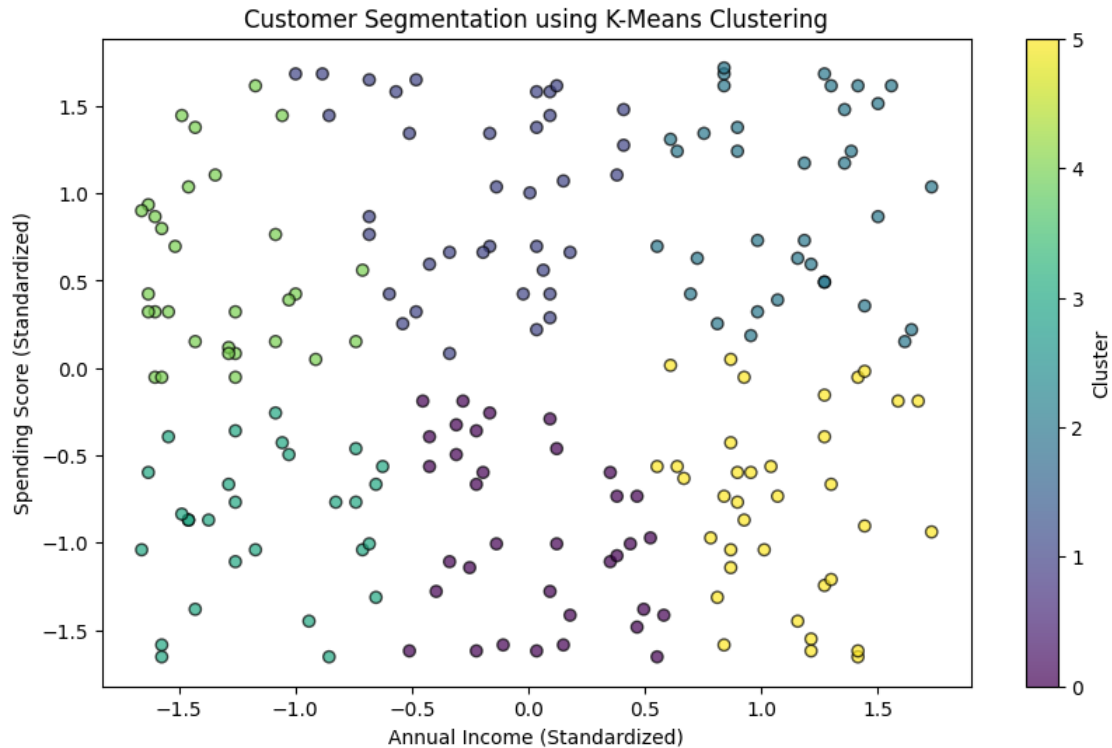
# Scatter plot of the clusters
plt.figure(figsize=(10, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=df['Cluster'], cmap='viridis',
            alpha=0.7, edgecolors='k')
plt.xlabel("Annual Income (Standardized)")
plt.ylabel("Spending Score (Standardized)")
plt.title("Customer Segmentation using K-Means Clustering")
```

```
plt.colorbar(label="Cluster")
plt.show()
```

	Customer ID	Annual Income	Spending Score
0	1	117	13
1	2	66	32
2	3	107	71
3	4	29	59
4	5	121	86
..	...	...	...
195	196	123	49
196	197	29	17
197	198	57	44
198	199	43	92
199	200	50	30

[200 rows x 3 columns]





```
[ ]: #plot cluster scatter plot
plt.figure(figsize=(10,6))
kmeans=KMeans(n_clusters=5,random_state=42)
kmeans.fit(X_scaled)
```

```
[ ]: KMeans(n_clusters=5, random_state=42)
```

<Figure size 1000x600 with 0 Axes>

```
[ ]: # Import for Hierarchical Clustering
import scipy.cluster.hierarchy as sch
from sklearn.cluster import KMeans, AgglomerativeClustering
import matplotlib.pyplot as plt

# Plot Dendrogram
plt.figure(figsize=(12, 6))
linked = sch.linkage(X_scaled, method='ward')
sch.dendrogram(linked)
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.title("Dendrogram for Hierarchical Clustering")
plt.show()
```

```

# Apply Agglomerative Clustering with chosen number of clusters (assume 4)
optimal_hc_k = 4
hc = AgglomerativeClustering(n_clusters=optimal_hc_k, metric='euclidean',
    linkage='ward') # Replaced 'affinity' with 'metric'
df['Hierarchical_Cluster'] = hc.fit_predict(X_scaled)

# Scatter plot for Hierarchical Clustering
plt.figure(figsize=(10, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=df['Hierarchical_Cluster'],
    cmap='coolwarm', alpha=0.7, edgecolors='k')
plt.xlabel("Annual Income (Standardized)")
plt.ylabel("Spending Score (Standardized)")
plt.title("Customer Segmentation using Hierarchical Clustering")
plt.colorbar(label="Cluster")
plt.show()

```

